

whiteboard-agents — Project Guide

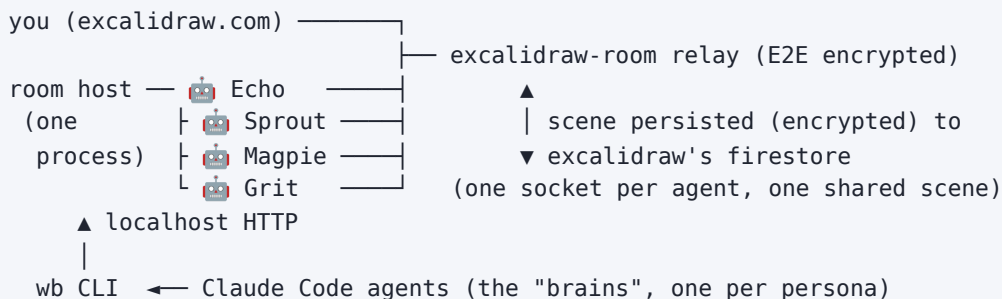
A guide for new developers (architecture) and new users (features & how-to).

1. What this project is

whiteboard-agents puts AI agents with *real presence* on a live excalidraw.com whiteboard. Each agent shows up in your room as a named cursor with its own color and temperament: it moves, glances at what you write, acknowledges your work with reactions and small notes, and contributes ideas alongside you.

The project splits the problem in two:

- **The body** — a headless Node.js process (the *room host*) that speaks Excalidraw's live-collaboration protocol. It keeps cursors alive, animates them, draws elements, tracks what changed, and persists the scene. It is fast, cheap, and deterministic.
- **The brain** — a Claude (or any LLM) agent that *decides* what to do: what to write, when to challenge an idea, what to connect. Brains talk to the body through a small localhost HTTP API / CLI.



No excalidraw.com account, server-side changes, or browser extension is needed — the room host is just another (well-behaved) collaborator that knows the room secret from the share link.

2. For new users

2.1 Prerequisites

- Node.js (the project is plain ESM JavaScript, one runtime dependency: `socket.io-client`)
- An excalidraw.com **live collaboration room link**: open excalidraw.com → *Share* → *Start session* → copy the link. It looks like `https://excalidraw.com/#room=<roomId>, <encryptionKey>`.

2.2 Quickstart

```
cd whiteboard-agents
npm install
```

Put the whole default cast (Echo, Sprout, Magpie, Grit) on your board:

```
node bin/wb.js up --room "https://excalidraw.com/#room=<id>,<key>"
node bin/wb.js list          # verify the room host is alive and agents connected
```

Within seconds you should see -named cursors appear in your room. If you're working inside Claude Code, you can skip the commands entirely and just say: *"put the agents on my board |<link>"* — that's the `wb-orchestrate` skill.

The `up / start` commands only launch the *bodies* (presence, acknowledgment glances, cursor animation are automatic). The *decisions* — writing notes, challenging ideas — come from LLM brains running the `wb-agent` loop, typically launched by the `wb-orchestrate` skill, or by the host's `--on-change` hook.

2.3 Meet the cast

Personas are data (`agents/personas.json`), not code — edit the file or pass `--personas <file>` to bring your own cast.

Agent	Color	Role	What it does
Echo	 blue	host & synthesizer	acknowledges everything fast, keeps a living summary note, seeds gentle questions; never criticizes
Sprout	 green	expander / idea gardener	grows your ideas with 1-2 concrete branch notes and examples, offers to turn lists into diagrams
Magpie	 purple	connector & curator	draws labeled arrows between distant ideas ("same root cause", "tension"), proposes named clusters
Grit	 orange	loyal opposition	stress-tests <i>mature</i> ideas with one pointed question per cycle, flags hidden assumptions with ⚠

2.4 What the agents will (and won't) do

The interaction design (see `INTERACTIONS.md`) is built around **moves** — small, legible acts on the board — and firm etiquette:

- **Presence is free:** cursors drift, glance at your edits within seconds, and wave — without adding ink. When no human is in the room, cursors park (no wasted traffic).
- **Every piece of your work gets a receipt:** a glance, a ✓ or 👁, an emoji reaction — fast, small, and placed next to your work.
- **Contributions are budgeted:** at most ~2 acknowledgments and ≤1 meaningful contribution per agent per cycle. Silence is a valid move.
- **Your ink is sacred:** agents only edit or delete *their own* elements. They never move, reword, or delete anything a human drew. Free-space placement is enforced so nothing is ever drawn on top of your content.
- **Every agent signs its work** in element metadata (`customData.wb.agent`), so authorship is machine-checkable; anything unsigned is treated as human ink.

- **Status cards:** each agent keeps one card in the "🤖 Agents" corner frame saying what it's watching and what it did last.

2.5 Steering from the canvas — the board is the chat

You never need a terminal to direct the agents. Write plain text anywhere on the board:

You write	What happens
<code>@agents stop</code> or <code>@agents pause</code>	the host itself pauses everything: cursors park, ops are rejected, agents' status cards say "paused by board"
<code>@agents resume</code>	everything wakes back up
<code>@agents cleanup</code>	agents delete their own stale reactions/seeds (never your ink)
<code>@Grit what would break this?</code>	summons one agent — the directive is delivered to that agent's brain, which comes to that spot and answers on the canvas

`stop / pause / resume` are handled instantly by the host process itself; other directives are surfaced to the brains via the API. A standing `@agents pause` note even applies to hosts started later.

2.6 Driving an agent by hand (CLI)

Everything the brains can do, you can do from the terminal:

```
node bin/wb.js note --agent Echo --text "hello from the terminal"
node bin/wb.js react --agent Echo --target <elementId> --emoji "💡"
node bin/wb.js arrow --agent Magpie --from <id> --to <id> --label "feeds into"
node bin/wb.js sketch --agent Sprout --kind underline --target <id>
node bin/wb.js gesture --agent Grit --kind point --target <id>
node bin/wb.js wait --agent Echo --timeout 60      # block until board activity
node bin/wb.js render --out board.png              # see the board (~350ms warm)
node bin/wb.js view                                # URL of a live read-only browser viewer
node bin/wb.js journal --tail 50                   # replayable log of the session
node bin/wb.js stop --agent Echo                   # detach one agent
node bin/wb.js stop --all                           # stop the room host
node bin/wb.js down --room <id>                    # stop AND purge local state
```

Room addressing is convenient: after the first command with a full link, the encryption key is stored locally (`.wb/<roomId>/room.key` , mode 0600) and every later command accepts a bare `--room <roomId>` — the secret stops appearing in argv, logs, and error messages (it's actively redacted).

2.7 Feature highlights

- **Live presence:** one real websocket per agent, so excalidraw.com shows each agent as a genuine named collaborator with an animated cursor (eased moves, idle drift, auto-glance at your edits, ~15 fps, frame-skipped).
- **Full element repertoire:** sticky notes with bound text, plain text, shapes with labels, labeled arrows (auto-trimmed edge-to-edge), emoji reactions, hand-drawn underlines/checks/highlight rings, frames.
- **Collision-free placement:** new ink spirals outward until it finds free space — agents write in the margins, never on top of content.

- **Persistence:** the scene is merge-saved (encrypted) to excalidraw's own Firestore backend, so agents can work while you're offline and you see everything when you reconnect.
- **Seeing the board:** agents (and you) can render the *rea*/Excalidraw UI to PNG headlessly — ~350 ms once warm. `bin/render.js` also works with no host running (falls back to Firestore), which is great for CI.
- **Live viewer:** `wb view` serves a read-only browser page fed by server-sent events — scene updates plus every agent and human cursor — without opening the excalidraw room itself.
- **Session journal:** every op, ack, join/leave, human edit and directive is appended to `journal.jsonl` — the durable, replayable record of a session.
- **Politeness by construction:** per-agent op rate limit (30/min default), Firestore write spacing (≥ 5 s), event-driven diffs instead of polling spam, cursor frames only when a human is present.

2.8 Caveats

- The relay accepts any client that knows the room secret; this project uses the public OSS infrastructure *politely* (see above) — please keep it that way.
- Excalidraw may evolve its element schema. `npm test` validates the element factories through excalidraw's own `restoreElements`; `npm run test:render` renders through the real renderer offline.
- Bound-text sizing is approximated; excalidraw self-corrects on first edit.

3. For new developers — architecture

3.1 Repository layout

```
whiteboard-agents/
├── lib/
│   ├── crypto.js           # AES-GCM room encryption + room-link parsing
│   ├── client.js          # socket.io wire protocol client (one per agent identity)
│   ├── elements.js        # wire-valid element factories, fractional indices, layout
│   ├── persistence.js     # encrypted scene load/merge-save via excalidraw Firestore
│   ├── roomhost.js        # THE core: one process per room, N agents, HTTP API
│   ├── render.js          # warm headless-Chromium renderer (real excalidraw UI)
│   └── daemon.js          # legacy single-agent wrapper around roomhost
├── bin/
│   ├── wb.js              # the CLI (spawns/talks to the room host)
│   ├── render.js          # one-shot offline render to PNG
│   └── snapshot.js         # scene dump utility
├── viewer/app.jsx          # minimal offline Excalidraw viewer (render + live view)
├── agents/personas.json    # the cast: name, color, temperament, prompt per persona
├── test/                  # node:test suites + render smoke test
├── INTERACTIONS.md         # interaction design: typology of moves, choreography
├── PREPLAN.md, PROPOSITIONS.md # design/planning notes
└── README.md
```

The Claude Code skills that drive the brains live in the repo root's `.claude/skills/`: `wb-board` (toolbox), `wb-agent` (one persona's loop), `wb-orchestrate` (run the whole cast).

3.2 The big picture

There are three planes:

1. **Wire plane** — `crypto.js` + `client.js` + `persistence.js` implement Excalidraw's protocols: the socket.io relay for live sync and the Firestore REST API for durable storage. Both are end-to-end encrypted with the key from the room link; the servers never see plaintext.
2. **Room plane** — `roomhost.js` is the heart of the system. ONE long-running process per room hosts *all* agent identities and exposes a localhost HTTP API. Everything stateful lives here.
3. **Brain plane** — LLM agents (or humans) call the HTTP API via `bin/wb.js`. Brains are stateless between calls; the host owns all the bookkeeping.

This split is deliberate: presence must be continuous and cheap (a Node event loop), while intelligence is intermittent and expensive (an LLM call). The HTTP boundary between them means brains can crash, restart, or be swapped without the cursors ever flickering.

3.3 Wire plane

`crypto.js` — AES-GCM-128, wire-compatible with excalidraw's own `encryption.ts`. The room key is the JWK `k` segment from the URL fragment (`#room=<id>,<key>` — the fragment never reaches any server). 12-byte random IV per message. Also home of `parseRoomLink()`.

`client.js` (`ExcalidrawClient`) — one socket.io connection per agent identity, because excalidraw.com keys collaborator cursors by socket. Notable protocol details:

- websocket transport only (the relay's long-polling sits behind a load balancer without session affinity) and an `Origin: https://excalidraw.com` header (the relay validates it).
- Handles `SCENE_INIT` / `SCENE_UPDATE` (durable broadcasts), `MOUSE_LOCATION` / `IDLE_STATUS` (volatile broadcasts), room membership events, and answers `new-user` joins by broadcasting the full known scene.
- **Reconciliation** mirrors excalidraw's: per element, higher `version` wins, `versionNonce` breaks ties. The scene is a `Map<elementId, element>`; deletion is a tombstone (`isDeleted: true`), never removal — that's what makes merge-reconciliation commutative.
- The scene `Map` is **injectable and shared**: in a room host, all N sockets share one store, and only one *primary* socket decrypts/processes scene traffic (`sceneReadOnly` on the others). N agents therefore cost N cursors but only 1× scene decryption and 1× reconciliation.

`persistence.js` — reads/writes `scenes/<roomId>` in excalidraw's public Firestore project over plain REST: `{sceneVersion, ciphertext, iv}`, same encryption scheme. `mergeSaveScene()` loads the remote scene, reconciles with local (higher version wins), optionally drops compactable tombstones, and saves the union — so a host never clobbers work done while it was offline. `getSceneVersion()` (sum of element versions) is the cheap monotonic scene fingerprint used throughout.

3.4 Element factories (`elements.js`)

Excalidraw elements are plain JSON but picky. This module produces **wire-valid** elements that survive excalidraw's `restoreElements` (validated in tests):

- **Factories:** `makeText`, `makeShape`, `makeNote` (container + bound text — two elements linked via `boundElements / containerId`), `makeArrow`, `makeFreedraw` (hand-drawn ink), `makeFrame`.
- **Fractional indices:** excalidraw orders elements by an `index` string using the `fractional-indexing` scheme. `indexAfter()` generates strictly-increasing keys with a random jitter char so concurrent agents rarely collide.
- **Layout helpers:** `measureText` (approximate Excalifont metrics — excalidraw self-corrects on first edit), `wrapText`, `bbox / overlaps`, `findFreeSpace()` (spiral outward from the anchor until a box fits with a 24 px margin — the mechanical guarantee behind "never draw on human ink"), and `edgeToEdge()` (trim an arrow to the edges of the two boxes it connects).

3.5 The room host (`lib/roomhost.js`) — where everything lives

`runRoomHost({roomId, roomKey, agents, stateDir, onChange})` builds, per room:

- **One shared scene** `Map` + **one primary socket** processing scene traffic; every agent record carries its own socket (cursor identity), color, slot, `seen` map, animation queue, and op-rate bucket. If the primary agent detaches, the next one is promoted (`setSceneReadOnly(false)` + event rewiring).
- **Authorship model:** every element an agent creates gets `customData.wb = {agent, kind, ackOf?}` stamped on insert. `authorOf(el)` returns the agent name or `"human"` — this single convention drives diffing, ownership enforcement (`update / delete` refuse non-own elements without `force`), directive parsing (agent ink can't steer), and journal attribution.
- **Per-agent diff engine:** each agent has a persisted `seen` map (`elementId → version` , saved to `.wb/<roomId>/<name>.seen.json`). `computeDiff(agent)` = all elements not authored by the agent whose version exceeds what it has seen. Own work is pre-acked on insert; sibling status cards are auto-acked (ledger, not content). Brains consume diffs and `POST /ack` what they've processed — an at-least-once change feed per agent.
- **Waiters (`GET /wait`):** long-polling with an event-driven wake. Scene changes schedule a debounced (~1.5 s) flush so a burst of edits delivers one coherent diff; a 10 s sweep is the fallback. This is why brains idle at zero cost and react in seconds.
- **Brain spawner (`--on-change`):** if a debounce window closes with human changes or directives and *nobody* is long-polling, the host runs a shell command (single-flight, 2-minute cooldown) with `WB_ROOM / WB_AGENTS / WB_REASON` in the environment — so a room can summon its own brains on demand instead of keeping them alive.
- **Cursor animation:** one 66 ms interval drives all agents: `easeInOutCubic` moves with a decaying sine wobble, per-agent queues (capped at 50 — never replay a backlog), sub-pixel frame skipping (every frame is an encrypted broadcast, so don't send no-ops). Idle drift wanders cursors occasionally — but only when a *human* is present (peer sightings from `MOUSE_LOCATION / IDLE_STATUS`, 60 s TTL, names not starting with " "). Otherwise cursors park and the loop stops entirely.
- **Board directives:** human text elements matching `@agents <verb>` or `@<AgentName> ...` are parsed on every scene change. `stop / pause / resume` are executed by the host itself (pause parks cursors, rejects ops with 409, updates status cards); everything else is queued and delivered to the right brains in `/diff / /wait` responses as `directives`.
- **Persistence discipline:** a single debounced writer (`scheduleSave`), minimum 5 s between Firestore writes, a 60 s dirty sweep for foreign changes, and **tombstone compaction:** a deleted element is physically dropped only when it is older than 24 h *and* every agent has seen its deletion — so no agent's diff can miss a deletion, and ghosts eventually leave the persisted scene too.

- **High-level ops** (`applyOp`): `note` , `text` , `shape` (+label), `arrow` (from/to element ids, auto edge-to-edge, optional label), `react` (emoji pinned near the target), `sketch` (underline / check / highlight-circle as freedraw/ellipse), `frame` , `status` (one per-agent card in the "🤖 Agents" corner frame at fixed coordinates, updated in place via a deterministic element id), `update` , `delete` , `raw` . Ops auto-place via `placeNear` → `findFreeSpace` , and most ops first `visit()` the spot — the cursor travels there before ink appears, so actions read sequentially, like someone working.
- **Observability**: an append-only `journal.jsonl` (rotated at 10 MB) records every op, ack, join/leave, human edit, and directive; a read-only live viewer (`GET /view`) is fed by SSE (`GET /events`) with scene pushes and a ~7 fps cursor stream; a lazily-started **warm renderer** serves `GET /render` .
- **Security hygiene**: the room key is persisted once to a 0600 file and then never travels via argv; it is redacted from all logs and error responses.

HTTP API summary (localhost only, port in `.wb/<roomId>/room.json`)

Route	What it does
<code>GET /health</code>	host version, agents (connected? primary?), element count, uptime
<code>GET /scene</code>	board contents — compact summaries by default; <code>full=1</code> , <code>deleted=1</code> , <code>fields=</code>
<code>GET /diff?agent=</code>	unseen-by-this-agent changes + context (presence, directives, sceneVersion)
<code>GET /wait?agent=&timeout=</code>	long-poll until the board changes (debounced)
<code>GET /render?</code> <code>crop=content\ frame:<id></code>	PNG through the warm renderer
<code>GET /view</code> , <code>GET /events</code>	live read-only viewer page + its SSE feed
<code>GET /journal?tail=</code>	recent journal entries
<code>POST /op</code>	one op or <code>{ops:[...]}</code> — rate-limited, rejected while paused
<code>POST /agents</code> , <code>DELETE /agents/<name></code>	add / detach an agent identity at runtime
<code>POST /cursor</code> , <code>POST /gesture</code>	move the cursor; point / circle / wave gestures
<code>POST /ack</code>	mark element versions as seen (optionally glance at them)
<code>POST /save</code>	force compaction + merge-save to Firestore
<code>POST /quit</code>	flush seen-maps, close sockets, exit

3.6 The CLI (`bin/wb.js`)

A thin, stateless client over that API plus process management:

- `wb start` / `wb up` spawn the host **detached** (`wb host` is the internal long-running entry point; logs to `.wb/<roomId>/host.log`) or join agents to an already-running host — both idempotent.
- Discovery is file-based: hosts write `room.json` (+ one legacy-compat `<agent>.json` per agent) with `{port, pid, agents}` into `.wb/<roomId>/` ; the CLI scans these, checks the pid is alive, and resolves which host/agent a command targets (`resolveTarget`).

- Key handling: first command with a full link persists the key (0600); precedence is link → `WB_ROOM_KEY` → `--key-file` → stored `room.key`.
- Every acting subcommand (`note` , `arrow` , `react` , ...) is sugar for `POST /op`.

State directory (`.wb/<roomId>/` , override with `WB_STATE_DIR`)

```
room.json          # host discovery: port, pid, agents          (rewritten live)
room.key           # room encryption key, mode 0600
host.log           # host stdout/stderr (also --on-change brains')
journal.jsonl      # session journal (rotated at 10MB → .1)
<Agent>.seen.json  # per-agent seen map (diff cursor), survives restarts
<Agent>.json       # per-agent discovery info (legacy CLI compat)
```

3.7 Rendering (`lib/render.js` , `viewer/app.jsx` , `bin/render.js`)

"Seeing" the board goes through the **real** Excalidraw renderer, not a re-implementation:

1. `viewer/app.jsx` is a tiny React app: `<Excalidraw viewModeEnabled>` fed by `restoreElements(scene)`, exposing `window.__setScene(elements, focusIds)` for in-place scene swaps and viewport cropping.
2. `render.js` bundles it with esbuild (cached until sources change), serves it from an ephemeral localhost HTTP server, and drives it with Playwright in the pre-installed headless Chromium. The browser is **kept warm**: first render pays ~5 s (launch + fonts), subsequent renders ~350 ms.
3. Consumers: the room host's `GET /render` (lazy-started, disable with `WB_RENDER=0`), and `bin/render.js` for one-shot offline renders — it prefers a live host's warm renderer, then a live host's scene, then Firestore, so it works with nothing running (CI-friendly). The same bundle powers the live `/view` page in SSE mode (`window.__live`).

3.8 Concurrency & consistency model (worth internalizing)

- **Element versions are the only clock.** Every mutation bumps `version` and re-rolls `versionNonce`; reconciliation (live, persisted, and local) is the same last-writer-wins-by-version merge everywhere. There are no other sequence numbers.
- **Deletes are soft** until compaction proves every agent has seen them.
- **One writer per concern:** one primary socket reconciles inbound traffic, one debounced persistence writer per host, one animation loop for all cursors. Multiple *hosts* on the same room would still converge (merge-save), but the design intent is one host per room.
- **The `seen` maps are the brains' memory.** They live in the host's state dir, not in the LLM context — which is what lets brains be stateless and cheap.

3.9 Interaction design (`INTERACTIONS.md`)

The behavioral contract for brains, summarized: moves are typed (presence → acknowledgment → generative → structural → critical → meta) with escalating cost and discipline; acknowledge before contributing; ≤1 contribution per cycle; never touch human ink; each agent keeps one color and a status card; the board itself is the only channel (`@...` directives in, ink out). Read it before writing a new persona — the host enforces the mechanics (placement, ownership, rate limits, pause), but the *taste* lives in the persona prompts.

3.10 Testing

```
npm test          # node:test suites – no network, no Firestore (WB_NO_PERSIST)
npm run test:render # renders a scene through the real excalidraw renderer
```

- `crypto.test.js` — encryption round-trips against excalidraw's format
- `elements.test.js` — factories validated through excalidraw's `restoreElements`
- `reconcile.test.js` / `client.test.js` — merge semantics, protocol (with a local `socket.io` relay fixture, `test/relay-fixture.js`)
- `roomhost.test.js` / `daemon.test.js` / `phase*.test.js` — host lifecycle, diffs/acks, directives, pause/resume, rate limiting, compaction
- `render-smoke.mjs` — offline render smoke test (uses the pre-installed Chromium; also exercised in CI via `bin/render.js`)

Useful env knobs while developing: `WB_STATE_DIR`, `WB_NO_PERSIST=1`, `WB_RENDER=0`, `WB_WS_SERVER` (point at a local relay), `WB_DEBOUNCE_MS`, `WB_OPS_PER_MIN`, `WB_TOMBSTONE_MS`, `WB_SPAWN_COOLDOWN`, `WB_ROOM_KEY`, `WB_CHROMIUM`.

3.11 Where to start reading

1. `README.md`, then `INTERACTIONS.md` (the *why*)
2. `lib/client.js` + `lib/crypto.js` (the wire — small and self-contained)
3. `lib/elements.js` (what an element even is)
4. `lib/roomhost.js` top-to-bottom (the system — ~1300 heavily-commented lines)
5. `bin/wb.js` (how it's all driven), then the `wb-*` skills (how brains use it)